

National University of Computer and Emerging Sciences

0144

FAST School of Computing

Fall-2023

Islamabad Campus

CS-2001: Data Structures
(BS-SE)

Monday, 25th March, 2023

Course Instructors

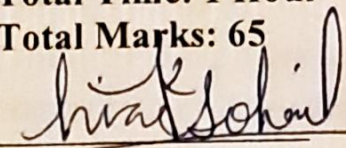
Ms. Maheen Arshad, Ms. Shahela Saif, Mr. Asif Muhammad
Malik

Serial No:

Sessional Exam-I

Total Time: 1 Hour

Total Marks: 65


Signature of Invigilator

Abdullah Daoud
Student Name

22I-2626
Roll No.

E
Section

Abdullah Daoud
Signature

DO NOT OPEN THE QUESTION BOOK OR START UNTIL INSTRUCTED.

Instructions:

1. Attempt on question paper. Attempt all of them. Read the question carefully, understand the question, and then attempt it.
2. No additional sheet will be provided for rough work. Use the back of the last page for rough work.
3. If you need more space write on the back side of the paper and clearly mark question and part number etc.
4. After asked to commence the exam, please verify that you have eight (9) different printed pages including this title page. There are a total of 3 questions.
5. Calculator sharing is strictly prohibited.
6. Use permanent ink pens only. Any part done using soft pencil will not be marked and cannot be claimed for rechecking.

	Q-1	Q-2	Q-3	Total

Question 1 [5 Marks]

Encircle the correct answer each carry [1 Mark]

1. Consider an implementation of unsorted singly linked list. Suppose it has its representation with a head pointer only. Given the representation, which of the following operation can be implemented in $O(1)$ time?
 - ☒ a) Insertion at first
 - b) Insertion at last
 - c) deletion at last
 - d) deletion at first
2. What would be the time complexity to insert an element at the front of the linked list (head is known)?
 - ☒ a) $O(1)$
 - b) $O(n)$
 - c) $O(n^2)$
 - d) $O(n^3)$
3. In the Selection Sort algorithm, how many comparisons are made in the first pass for an array of n elements?
 - a) n
 - ☒ b) $n - 1$
 - c) $n/2$
 - d) 0
4. In Selection Sort, what is the purpose of the outer loop?
 - a) It selects the minimum element in the list.
 - b) It swaps the adjacent elements.
 - ☒ c) It iterates through the entire list.
 - d) It checks if the list is already sorted.
5. In Selection Sort, what is the primary advantage of reducing the number of swaps?
 - a) It reduces the overall time complexity.
 - b) It makes the algorithm more stable.
 - ☒ c) It minimizes the number of comparisons.
 - d) It reduces the I/O operations.

Question 2 [30 marks]

1. What is the output of the following function for a Linked List having nodes 1->2->3->4->5->6? You also have to specify what exactly the function is doing. [10 marks]

```
void fun2(Node* head)
{
    if(head == NULL)
        return;
    cout << head->data << " ";

    if(head->next != NULL )
        fun2(head->next->next);
    cout << head->data << " ";
}
```

Output: 1 3 5 5 3 1

Explanation: The function is printing odd numbers recursively and then printing them again in reverse order

2. Calculate the big-Oh complexity for given code segment. [15 marks]

Statement		Cost	Times
int i = 1;	C1	1	1
while (i < n) {	C2	1	n

int j = 1;	C3	1	n
while (j <= i * i) {	C4	2	$\log n$
if (j % i == 0) {	C5	2	$\log n$
int k = j;	C6	1	n^2
while (k < i) {	C7	1	n^3
sum++;	C8	1	n^3
k++;	C9	1	n^3
}	C10	1	n^3
}	C11	1	n^2
j++;	C12	1	$\log n$
}	C13	1	$\log n$
i++;	C14	1	n
}	C15	1	n

Total Time Complexity: $T(n) = 2n \log n + 4n + 2n^2 + 4n^3$

Big-O value: $O(n^3)$

3. Given the following input array, perform sorting (Ascending order) using the selection sort algorithm. Fill the given table for each pass [10 marks]

	17	11	14	13	19	100	77	5	16
Before Sorting	17	11	14	13	19	100	77	5	16
Pass 1	17	11	14	13	19	16	77	5	100
Pass 2	17	11	14	13	19	16	5	77	100
Pass 3	17	11	14	13	5	16	19	77	100
Pass 4	17	11	14	13	5	16	19	77	100
Pass 5	17	11	5	13	14	16	19	77	100
Pass 6	17	11	5	13	14	16	19	77	100
Pass 7	17	5	11	13	14	16	19	77	100
Pass 8	5	17	11	13	14	16	19	77	100
Pass 9	5	11	13	14	16	17	19	77	100

Question 3 [30 marks]

1. Write a C++ function to check if a given singly linked list is palindrome or not. [10 marks]

Input: R->A->D->A->R->NULL

Output: Yes

Input: C->O->D->E->NULL

Output: No

```

void Is_Palindrome (Node* head)
{
    char arr1[50];
    char arr2[50];
    bool flag = false;
    int i = 0;
    Node* temp = head;
    while (temp != NULL)
    {
        char arr1[i] = temp->data;
        temp = temp->next;
        i++;
    }
    reverse Nodes (temp) // Function to reverse
        recursively links of a list
    Node* current = temp;
    while (current->next != NULL)
    {
        head = current;
        return current;
    }
    else cout << "Not a palindrome" << endl;
}

Node* nextNode = reverseNodes (current->next);
current->next = NULL;
return current;
}

int j = 0;
Node* temp = head;
while (temp != NULL)
{
    arr2[j] = temp->data;
    temp = temp->next;
    j++;
}

for (int i = 0; i < size; i++)
    for (int j = i + 1; j < size; j++)
        If (arr1[i] == arr2[j])
            cout << "list is palindrome" << endl;
            else cout << "Not a palindrome" << endl;
}

```

2. Alladin has been exploring a magic cave by himself since his Genie left him. The cave is full of wonderful things but is also riddled with monsters. While on the journey Alladin collects the gold coins he receives. He also gathers the weapons he finds but only if he has more than 2 gold coins since each weapon costs 2 coins. Every time he encounters a monster he uses one of his weapons. However, every weapon can be used only once. Alladin has memory problems. He only remembers the newest item he collected and does not think of the one before it till newest is used.

Devise a game for this scenario where you are required to provide only 2 functions – The one where weapons are collected and one where monsters are encountered. [10 marks]

```
void WeaponCollection (int weaponID coins) {
```

// here weapons that are found will be collected. Weapon IDs will be stored for later use.

// Only the latest weapon is accessible

```
const int MAX_SIZE = 101; } declared
int arr weaponID [MAX_SIZE]; } global
int top = -1;
```

IF (coins < 2)
cout << "Not
Enough coins! \n";

IF (top == MAX_SIZE - 1)

cout << "Weapon Collection is Full! \n"; ^{ok}

return;

~~top++;~~

^{weaponID}

IF (coins >= 2)

{ top++;

} arr[top] = weaponID; ✓

```
void MonsterFight (bool encounter)
```

```
{
```

// When we encounter a monster, we use one of our weapons

// Weapon can be used just once

// Previous weapon is accessible once latest is used

```
const int MAX_SIZE = 101
```

```
int arr [MAX_SIZE];
```

```
int top = -1;
```

cout << "Next Weapon:"
<< arr[top] << endl;

IF (top == -1)

{ cout << "You don't have weapons! \n";

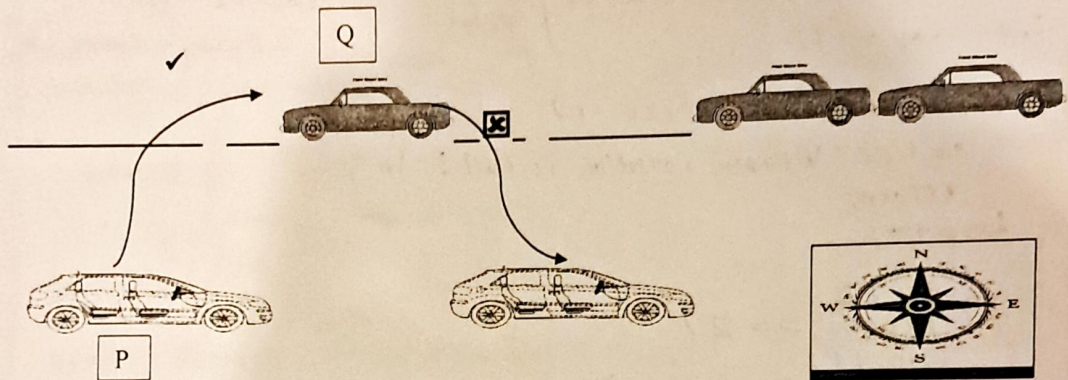
return;

IF (encounter == true)

{ cout << "Weapon Used:" << arr[top] << endl;

} top--;

3. Let us consider a non-empty zero-indexed array "A" consisting of N characters being used to represent consecutive cars on a road. Each element of the array contains only either E/W characters; E represents a car travelling towards east, W represents a car travelling towards west. As an example, the following road situation can be modeled as below:



To consider array A for the given situation is such that:
A[0]=E, A[1]=W, A[2]=E, A[3]=W, A[4]=W

We say that two cars cross if any next car in the array is going in the opposite directions. More formally we say that a pair of cars, given $0 \leq p < q < n$ and p is travelling to the east and q is travelling to the west. Write a generic C++ function to count crossing cars. If we have five pairs of passing cars: (0,1) (0,3) (0,4) (2,3) (2,4) the method should return 5 as explained above. [10 marks]

```
void int countCrossingCars (int A[], int size)
{
    int count = 0;
    for (int i = 0; i < size; i++)
    {
        if ((A[i] == 'E' && A[i+1] == 'W') ||
            (A[i] == 'W' && A[i+1] == 'E'))
        {
            count++;
        }
    }
    cout << "Number of Crossing Cars = ";
    return count;
}
```